Apellidos: .....Nombre:.....Grupo:.....DNI:.....

Examen EP2 (temas 8-14)

Duración: 2 horas 30 minutos. La solución de cada ejercicio se tiene que escribir en el espacio reservado para ello en el propio enunciado. No podéis utilizar calculadora, apuntes, etc. La solución del examen se publicará en Atenea el 23 de diciembre por la tarde y las notas antes de las 23:59 del 2 de enero.

Ejercicio 1 (1.5 puntos) El programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina situando la sección de datos (.data) a partir de la dirección 0xC000 y justo a continuación la sección de código (.text).
Una vez ensamblado y cargado el programa en la memoria y antes de comenzar su ejecución:

```
.data
        N = 5
V1:     .word 0x0100, 0x0302, 0x0504
        .even
Res:    .space 2

.text
        MOVI    R1, lo(N)
        MOVHI   R1, hi(N)
        MOVI    R2, lo(V1)
        MOVHI   R2, hi(V1)
        MOVI    R3, lo(Sub)
        MOVHI   R3, hi(Sub)
        MOVI    R4, 0
        ADDI    R5, R2, 0
loop:   JALR    R7, R3
        LDB     R6,0(R2)
        ADD     R4, R6, R0
        ADDI    R2, R2, 1
        ADDI    R1, R1, -1
        BNZ     R1, loop
        ST      6(R5), R4
        …
sub:    IN      R0, Key_status
        BZ      R0, sub
        IN      R0, Key_data
        JALR    R7, R7
.end
```

a. ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes?

Loop = 0xC018 Res = 0xC006

Corrección: 0,1 cada una (binario)

b. ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa?

Instrucción	@memoria y contenido
`MOVHI R1, hi(N)`	Mem_w[0x C00A]=0x9300
`BNZ R1, loop`	Mem_w[0x C022]=0x83FA / 8BFA (si no hancanviat R5 x R1)

Corrección: 0,2 cada dirección y 0,2 cada contenido (total 0,8)

c. Una vez ejecutado el programa en el SISC Von Neumann, asumiendo que el usuario teclea un "5" cada segundo hasta que finaliza el programa, ¿Cuál es la dirección de la memoria donde ha escrito la instrucción (ST 6(R5), R4) y cuál es su contenido?

Mem_w[0x C006]=0x0009

Corrección: 0,2 dirección, 0,3 contenido

Ejercicio 2 (0,5 puntos) Indica el contenido de la ROM_Q+ en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado).

`ROM_Q+[0x00A]`	0x01	`ROM_Q+[0x02A]`	0x05
`ROM_Q+[0x22E]`	0x00	`ROM_Q+[0x035]`	0x10

Corrección: -0,2 por cada error (mínimo = 0)

Ejercicio 3 (1 punto) Completa la siguiente tabla ensamblando las instrucciones en ensamblador SISA o desensamblando las instrucciones en lenguaje máquina según sea necesario. Indica poniendo NA en la casilla aquellos casos en los que la instrucción no corresponda al lenguaje SISA.

Lenguaje máquina SISA (en hexadecimal)	**Lenguaje ensamblador SISA**
0x0000	AND R0, R0, R0
0x8F11	BNZ R7, 17 (también BNZ R7, 0x11)
NA	`SUBI R5, R5, 1`
0xA710	`OUT 16, R3`

Corrección: 0,25 por cada casilla (binaria)

Ejercicio 4 (2 puntos) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado).

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
F	`JALR R3, R5`	0xC260F0
Addr	`ST 0x10(R5), R3`	0x000030
Out	`OUT 0, R7`	0x080000
Bnz	`BNZ R2, 10`	0x800220

Corrección: 0,5 por cada casilla (binaria)

Ejercicio 5 (1 punto) Indicad qué cambios se producen en el estado del computador **SISC Von Neumann** después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale 0x2228, el contenido de todos los registros es 0x1234, el byte contenido en todas las direcciones pares de la memoria es 0x22 y el de todas las impares es 0x11 y el contenido del port PRINT_STATUS es 1. Utilizad la notación Ri=0x… para un cambio en el registro "i", MEMb[0x…]=0x... y/o MEMw[0x…]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida.

Corrección: 0,25 cada casilla (binario)

Instrucción a ejecutar	**Cambios en el estado del computador**
`JALR R5, R5`	PC=0x1234, R5 = 0x222A
`ST 6(R5), R3`	PC=0x222A, MEMw[0x123A] = 0x1234
`OUT PRINT_DATA, R3`	PC=0x222A, PORTOUT[PRINT_DATA]= 0x1234, PORTIN[PRINT_STATUS]=0
`BNZ R7, 0xFF`	PC=0x2228

Apellidos: .....Nombre:.....Grupo:.....DNI:.....

Ejercicio 6 (1 punto) Completad las filas y columnas de la siguiente tabla que representa un subconjunto de la palabra de control del SISC Von Neumann. Indicad solo los valores necesarios para realizar las acciones correspondientes a cada estado. Para el resto de valores, ya sean desconocidos o **no sean necesarios para ninguna acción del estado,** poned x.

Nodo / Estado	**Instrucción en IR** (en ensamblador)	**@A**	**@B**	**@D**	**F**	**N** (en hexadecimal)	**ADDR-IO** (en hexadecimal)
F	MOVHI R3, 0xFF	xxx	xxx	xxx	100	0x0002	0xXX
Addr	LDB R3, -1(R0)	xxx	xxx/011	xxx	100	0xFFFF	0xXX
Movhi	MOVHI R3, 16	xxx	xxx	011	010	0x0010	0xXX
Bnz	BNZ R7, 4	xxx	xxx	xxx	000	0xXXXX	0xXX

Corrección: 1-X*0,25 donde X es el número mínimo de filas y columnas que agrupan todos los errores.

Ejercicio 7 (1 punto) Especificad el camino crítico (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el tiempo de ciclo mínimo para que el computador SISC Von Newmann pueda ejecutar correctamente los siguientes estados. No debéis añadir ningún porcentaje de seguridad al cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

Tp**(ROM_Q+)** = 50 u.t.
Tp**(ROM_OUT)** = 60 u.t.
Tp**(MUX-2-1)** = 50 u.t.
Tp**(MUX-4-1)** = 100 u.t.
Tp**(REG)** = 150 u.t. // Tp de un registro.
Tp**(REGFILE)** = 250 u.t. // Tiempo de lectura/escritura de un registro.
Tp**(ALU-slow)** = 650 u.t. // Tp de ALU para operaciones lentas: ADD, SUB, CMP*.
Tp**(ALU-quick)** = 350 u.t. // Tp de ALU para operaciones rápidas: todas menos las anteriores.
Tacc**(64Kb MEMORY)** = 1000 u.t. // Tiempo de acceso (lectura o escritura) a la memoria. Fijaros que este tiempo incluye toda la lógica empleada para construir-la a partir de dos bloques de 32Kb.
Tp**(AND-2)** = Tp**(OR-2)** = 20 u.t.
Tp**(NOT)** = 10 u.t.

Desconocemos como se han implementado los bloques internamente (y podría ser diferente a como los hemos visto en clase). Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os facilitamos su esquema porque lo deberíais saber).

a. D (Decode) =

Estat – ROM_OUT – MUX4-1 (N) – MUX2-1 (Ry/N) - ALU-slow – R@

150 + 60 + 100 + 50 + 650 = 1010 u.t.

Corrección: camino Decode ➔ 0,25, Tiempo de ciclo bien (si camino bien) ➔ 0,25

b. Ld =

Estat -ROM_OUT – MUX2-1 (@R@/PC) – MEM – MUX4-1(P/I/L/A) – REGFILE

150 + 60 + 50 + 1000 + 100 + 250 = 1610 u.t.

Corrección: camino Ld ➔ 0,25, Tiempo de ciclo bien (si camino bien) ➔ 0,25

EP2 (22 diciembre 2025) **IC-25-26-Q1**

Apellidos: .....Nombre:.....Grupo:.....DNI:.....

Ejercicio 8 (2 puntos) Queremos empezar a adaptar nuestra arquitectura Von Newmann para que pueda trabajar con enteros de 32 bits. Para ello empezaremos añadiendo una instrucción que pone a cero 32 bits de memoria. La instrucción e la siguiente:

Sintaxis: SETZero-32 Ra **Codificación**: 1011 aaa x xxxx xxxx

Semántica: PC = PC+2; MEMw[Ra] = 0; MEMw[Ra+2] = 0

a. ¿Cómo modificaríais el **hardware de la UPG y el grafo de estados** para poder implementar esta nueva instrucción? Podéis modificar buses y añadir multiplexor(es), buses y señales de control, pero no podéis modificar ni los bloques ni el uso de las señales de control existentes. Describe las modificaciones con un dibujo de la parte relevante o mediante una explicación. Indicad también como cambiaria el grafo de estados.

Añadir un multiplexor en la entrada WR-MEM que elija entre RY o la constante 0. Conectamos el "0x0000" a la entrada 0 y RY a la entrada 1 (podría ser al revés y se debería ver en la tabla del apartado c)

F —(Xx)— D —(Bx)— S1 —(Bx/Xx)— S2 —(Bx/Xx)— S3 —(Xx)— F

Y cambiar para que F —(Xx)— D —(~~Bx~~Cx-Fx)— Nop

Corrección: 0,25 si grafo modificado correctamente, 0,25 si HW añadido correctamente

b. Indicad las acciones que se deberán ejecutar en cada uno de los estados de la nueva instrucción (**Sin incluir Fetch y decode**). La tabla adjunta tiene el número suficiente de filas, incluso es posible que no sean todas necesarias.
Corrección: 0,75-X*0,25 donde X es cada estado incorrecto (binario)

Estado	Acciones
S1	R@ = RX; RX = Ra
S2	MEMw[R@] = 0; R@ = RX + 2
S3	MEMw[R@] = 0

c. Indicad el contenido de las filas de la ROM_OUT que sea preciso modificar (sin incluir Fetch y decode). La tabla adjunta tiene el número suficiente de filas, incluso es posible que no sean todas necesarias. También hemos añadido un par de salidas nuevas por si las necesitas. De nuevo, puede que no las necesites todas.
Corrección: 0,75-X*0,25 donde X es el número mínimo de filas y columnas que agrupan todos los errores.

@ROM	New1	New2	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A1	P/I/L/A0	OP1	OP0	MxN1	MxN0	MxF	F2	F1	F0	Mx@D1	Mx@D0
		x	0	0	0	0	0	0	0	x	x	x	0	x	x	x	1	0	x	x	1	0	0	0	x	x
		0	0	0	1	0	0	0	0	0	1	x	0	0	x	x	0	0	1	1	1	1	0	0	x	x
		0	0	0	1	0	0	0	x	0	1	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x